# DMS System Security & Validation Reference

Internal technical reference for backend validation,
frontend integration, sanitization, SQL safety, and exception handling.

Document version: 1.8
Last updated: July 2026
Scope: Backend validation, data sanitization, SQL injection protection, exception handling, and frontend integration - module by module.

Generated from backend/docs/SECURITY_AND_VALIDATION.md

# DMS System Security & Validation Reference

## 1. Target behavior (all modules)

Every user-facing write operation should follow this pattern:

```
??????????????     ??????????????????????     ??????????????????????     ??????????????????
?  Frontend   ?     ?  Frontend        ?     ?  Backend        ?     ?  Backend     ?
?  Validate   ? ??? ?  Disable submit  ? ??? ?  Validate       ? ??? ?  Sanitize +  ?
?  on change  ?     ?  when invalid    ?     ?  in controller  ?     ?  parameterized?
??????????????     ??????????????????????     ??????????????????????     ?  SQL + service?
                                                                         ??????????????????
                                                                                 ?
                                                                                 ?
                                                                       ??????????????????????
                                                                       ? errorHandler ->   ?
                                                                       ? { message,       ?
                                                                       ?   errors[] }     ?
                                                                       ??????????????????????
```

| Layer | Responsibility | Can be bypassed? |
|---|---|---|
| Frontend validation | UX - early feedback, highlight fields | Yes (devtools, API tools) |
| Disabled submit button | UX - prevent accidental submit | Yes |
| Backend validation | Required - security & data integrity | No |
| Sanitization | Clean text before storage/search | No (in service/model path) |
| SQL parameterization | Prevent injection | No |

Rule: Never rely on frontend or disabled buttons alone. Backend always returns structured errors on invalid input.

### Standard API error response

```
{
  "success": false,
  "message": "Validation failed",
  "errors": [
    { "field": "email", "message": "Provider email is required" }
  ]
}
```

Frontend client (frontend/src/lib/auth/authApi.js):

```
throw new ApiRequestError(data.message, response.status, data.errors);
```

Shared frontend helpers (frontend/src/lib/apiErrorUtils.js):

Architecture flow: Frontend validates -> Backend validates -> Database (parameterized queries).

| Function | Purpose |
|---|---|
| shouldShowSubmitError(message, fieldErrors) | Hide generic banner when field errors exist |
| getApiErrorMessage(error, fallback) | Prefer first field message over generic API text |
| applyApiFieldErrors(error, fieldMap) | Parse API error into field errors + banner |
| mergeApiFieldErrors(error, setFieldErrors) | Merge into React state; returns banner text |

```
hasValidationErrors(validationErrors)
```

| |
|---|
| True when client validation object is non-empty |

## 2. Current system status (summary)

| Rating | Meaning |
|---|---|
| FULLY COVERED | Backend validates writes/queries; read-only or no forms |
| PARTIAL | Backend strong; frontend missing disable-on-invalid and/or errors[] field mapping |
| MISSING | Significant gap in frontend or backend for that module |

Architecture flow: Frontend validates -> Backend validates -> Database (parameterized queries).
Architecture flow: Frontend validates -> Backend validates -> Database (parameterized queries).

| Employees | Yes Strong | Yes Create/edit/suspend | Yes Create/edit/suspend | Yes Form modal + suspend modal |
|---|---|---|---|---|
| Orders | Yes Strong | Yes Main form + modals | Yes Main form + action modals | Yes Save + provider sync + modals |
| Invoices | Yes Strong | Yes Modals | Yes Create/X-ray/write-off | Yes Create + X-ray + write-off modals |
| Facilities | Yes Strong | Yes Create/edit/notes/docs | Yes Create/edit/notes/docs | Yes Main forms + note/upload modals |
| Providers | Partial Update only | Partial Via order page blur | N/A | Yes Order page maps sync errors |
| Payments | Yes Manual payment + list queries | Yes Basic | Yes Manual payment modal | Yes Manual payment modal |
| Settings | Yes Strong | Yes Profile/password | Yes Profile/password | Yes Profile/password |
| Notifications | Yes Query limit | N/A (read) | N/A | N/A |
| Reports | Yes Query parser | N/A (filters) | N/A | N/A |
| Stripe public | Yes Checkout | N/A | Partial Processing only | Yes publicPayApi.js |
| Activity log | Yes Service parse | N/A (read) | N/A | N/A |
| Dashboard | Yes Limit clamp | N/A (read) | N/A | N/A |

Conclusion: Backend validation remains the authoritative gate on all major write paths. Frontend now uniformly disables submit when client validation fails and maps API errors[] to form fields on the primary write surfaces (orders, invoices, facilities, employees, settings, payments, and order action modals).

## 3. Shared backend infrastructure

Architecture flow: Frontend validates -> Backend validates -> Database (parameterized queries).
Architecture flow: Frontend validates -> Backend validates -> Database (parameterized queries).
Architecture flow: Frontend validates -> Backend validates -> Database (parameterized queries).

| File | Purpose |
|---|---|
| src/validators/validationHelpers.js | Email, ISO date, money, SSN, positive IDs, max length |
| src/validators/*.js | Per-domain request validators |
| src/lib/facilityValidation.js | Facility & doctor rules |
| src/lib/reportQueryParser.js | Report query validation + sanitization |
| src/utils/sanitize.js | stripControlCharacters, sanitizeText, sanitizeSearchText, escapeHtml |
| src/utils/sqlSafety.js | escapeLike, likeContains, assertPositiveInt, assertIsoDate |
| src/utils/fieldLimits.js | DB column-aligned max lengths |

src/utils/serviceErrorUtils.js

rethrowServiceError, withTransaction, runNonCritical - shared service-layer error patterns

src/middleware/errorHandler.js

Global Express error middleware

src/middleware/uploadMiddleware.js

MIME allowlists, file size limits

src/app.js

```
express.json({ limit: "2mb" })
```

server.js

unhandledRejection, uncaughtException logging

### Validator return shape

```
{ valid: boolean, errors?: [{ field: string, message: string }] }
```

### Service-layer error handling

Architecture flow: Frontend validates -> Backend validates -> Database (parameterized queries).

| Helper | Use when | Behavior |
|---|---|---|
| rethrowServiceError(error) | Transaction catch, DB helpers, email send failures | Preserves ApiError; maps MySQL/JWT/FS/Stripe via errorMapper |
| runNonCritical(label, fn, logger) | Side effects (notifications, activity logs, milestone rollups, payment emails) | Logs warning; returns null - parent request still succeeds |
| asyncHandler.runSafely(label, fn) | Background jobs (invoiceReminderJob, employeeReactivationJob) | Logs error; returns null - job loop continues |

Controllers stay thin: no local try/catch; errors bubble to errorHandler.

## 4. Module reference (validation · sanitization · SQL · exceptions)

### 4.1 Auth - /api/auth

| Area | Details |
|---|---|
| Validation | authValidator.js - login (trim + max length), 2FA, resend 2FA, refresh, logout |
| Sanitization | Trim on identifier; password max 128 chars; 2FA code digits-only |
| SQL injection | Parameterized queries in AuthSession, Employee |
| Frontend | login/page.jsx - validates, disables submit, maps API errors[] to email/password; TwoFactorAuthModal.jsx maps code errors |

Exceptions handled

| Type | Name / code | HTTP | Client message |
|---|---|---|---|
| Custom | ApiError | 400/401 | Invalid credentials, validation failed |
| JWT | TokenExpiredError | 401 | Session expired. Please sign in again. |
| JWT | JsonWebTokenError | 401 | Invalid or expired session. Please sign in again. |
| JWT | NotBeforeError | 401 | Invalid or expired session. Please sign in again. |
| MySQL | ER_DUP_ENTRY | 409 | This record already exists. |
| Network | ECONNREFUSED, etc. | 503 | Database is temporarily unavailable. |

### 4.2 Employees - /api/employees

| Area | Details |
|---|---|
| Validation | employeeValidator.js - create, update, suspend (future datetime) |
| Query | validateMilestoneStatsQuery - ISO from/to |

Sanitization

sanitizeSearchText on list search

SQL injection

Employee - :named params, escapeLikePrefix

Frontend

EmployeeFormModal.jsx + SuspendEmployeeModal.jsx - validate, disable submit, map errors[]

Exceptions handled

| Type | Name / code | HTTP | Client message |
|---|---|---|---|
| Custom | ApiError | 400 | Validation failed (field errors) |
| Custom | ApiError | 400 | You cannot suspend your own account |
| Custom | ApiError | 400 | Admin accounts cannot be suspended |
| Custom | ApiError | 409 | This email is already in use |
| Custom | ApiError | 404 | Employee not found |
| MySQL | ER_DUP_ENTRY | 409 | This record already exists. |
| MySQL | ER_DATA_TOO_LONG | 400 | One or more fields exceed the allowed length. |

## 4.3 Orders - /api/orders

| Area | Details |
|---|---|
| Validation | orderValidator.js - create/update, facility, notes, workflow; provider email required |
| Actions | orderActionValidator.js - cancel, mail, CNR, certificate, copy letter, pickup, fax, batch scan, medical scan, remove medical records, medical record file type |
| Query | validateOrderNotesQuery, validateSearchQuery |
| Sanitization | sanitizeSearchText on filters; sanitizeText on notes |
| SQL injection | Parameterized queries; sortDir whitelist; escaped LIKE |
| Upload | PDF medical records; batch scan; multer limits |
| Frontend | orders/new/page.jsx - validates, disables save, maps API errors[]; action modals (OrderFaxModal, OrderPickupModal, SendCopyLetterModal, SendInvoiceEmailModal, OrderNotesModal, OrderAddNoteModal, OrderNotesListModal, OrderCancelModal) use apiErrorUtils |

Exceptions handled

| Type | Name / code | HTTP | Client message |
|---|---|---|---|
| Custom | ApiError | 400 | Validation failed (e.g. provider email required) |
| Custom | ApiError | 400 | Cancellation reason is required |
| Custom | ApiError | 400 | At least one recipient email is required |
| Custom | ApiError | 400 | A valid company email is required |
| Custom | ApiError | 400 | Invalid record type |
| Custom | ApiError | 400 | Cannot cancel a deleted order |
| Custom | ApiError | 404 | Order not found |
| Upload | MulterError | 400 | File upload error / size limit |
| Upload | "Unsupported file type" | 400 | Unsupported file type |
| MySQL | ER_* | 400/503 | Mapped via errorMapper |

## 4.4 Invoices - /api/invoices

| Area |
| --- |

**Details**

Validation

invoiceValidator.js - create/update, X-ray, send/resend, write-off, email arrays

Sanitization

sanitizeTrimOrNull on text in service

SQL injection

Parameterized; report ORDER BY from internal constants

Frontend

CreateInvoiceModal.jsx, CreateXrayInvoiceModal.jsx, WriteOffInvoiceModal.jsx - validate, disable submit, map errors[]

Exceptions handled

| Type | Name / code | HTTP | Client message |
|---|---|---|---|
| Custom | ApiError | 400 | Validation failed (amounts, dates, IDs) |
| Custom | ApiError | 400 | Invoice total must be greater than zero |
| Custom | ApiError | 400 | This invoice is already paid |
| Custom | ApiError | 404 | Invoice / order not found |
| MySQL | ER_DUP_ENTRY | 409 | This record already exists. |

## 4.5 Facilities - /api/facilities

| Area | Details |
|---|---|
| Validation | facilityValidator.js - create, update, resolve, resolve doctor, doctors, notes, document upload |
| Sanitization | sanitizeSearchText on search; sanitizeText on resolve-doctor name |
| SQL injection | escapeLikePrefix, parameterized queries |
| Upload | Document MIME/size limits |
| Frontend | facilities/new/page.jsx, facilities/[id]/info/page.jsx, FacilityAddNoteModal.jsx, UploadDocumentsModal.jsx - validate, disable submit, map errors[] |

Exceptions handled

| Type | Name / code | HTTP | Client message |
|---|---|---|---|
| Custom | ApiError | 400 | Validation failed (facility name, email, zip, etc.) |
| Custom | ApiError | 400 | Invalid document type |
| Custom | ApiError | 400 | A file is required |
| Custom | ApiError | 404 | Facility / doctor / note not found |
| Upload | MulterError | 400 | File size / upload error |
| MySQL | ER_ROW_IS_REFERENCED_2 | 400 | This record is in use and cannot be removed. |

## 4.6 Providers - /api/providers

| Area | Details |
|---|---|
| Validation | providerValidator.js - update (name, email format, zip, phone) |
| Query | validateSearchQuery |
| Sanitization | sanitizeSearchText, stripControlCharacters on text fields |
| SQL injection | likeContains, assertPositiveInt on limit |
| Frontend | No dedicated provider form; order page syncProviderFromForm maps API validation errors to provider fields on blur |

Exceptions handled

| Type | Name / code | HTTP | Client message |
|---|---|---|---|

Custom

ApiError

400

Provider company name is required

Custom

ApiError

400

Validation failed (email, zip, phone)

Custom

ApiError

404

Provider not found

## 4.7 Payments - /api/payments

| Area | Details |
|---|---|
| Validation | paymentValidator.js - manual payment; validatePaymentSearchQuery; validatePaymentListQuery (dates, orderId, limit 1-500) |
| Sanitization | sanitizeTrimOrNull on notes |
| SQL injection | Parameterized inserts/updates; list queries capped via parsePaymentListLimit |
| Frontend | ManualPaymentModal.jsx - validates, disables save when invalid, maps errors[] to check/date fields |

Exceptions handled

| Type | Name / code | HTTP | Client message |
|---|---|---|---|
| Custom | ApiError | 400 | orderId / check number / payment date required |
| Custom | ApiError | 400 | invoiceType must be regular or xray |
| Custom | ApiError | 400 | This invoice is already paid |
| Custom | ApiError | 404 | Order / invoice not found |

## 4.8 Settings - /api/settings

| Area | Details |
|---|---|
| Validation | settingsValidator.js - profile, password, notification booleans |
| Sanitization | Trim on profile fields |
| SQL injection | Parameterized Employee, EmployeeSettings |
| Frontend | Profile/password validate, disable save when invalid, map errors[] |

Exceptions handled

| Type | Name / code | HTTP | Client message |
|---|---|---|---|
| Custom | ApiError | 400 | Validation failed (firstName, email, password) |
| Custom | ApiError | 400 | Current password is incorrect |
| Custom | ApiError | 409 | This email is already in use |
| Custom | ApiError | 404 | User not found |

## 4.9 Notifications - /api/notifications

| Area | Details |
|---|---|
| Validation | validateNotificationQuery - limit 1-100 |
| Sanitization | sanitizeSearchText on optional search |
| SQL injection | Parameterized Notification model |
| Frontend | Read-only list + mark read |

Exceptions handled

| Type | Name / code | HTTP | Client message |
|---|---|---|---|
| Custom | ApiError | 404 | Notification not found |

## 4.10 Reports - /api/reports

| Area | Details |
|---|---|
| Validation | reportQueryParser.js - dates, page size, cursor, enums |
| Sanitization | sanitizeSearchText on search fields |
| SQL injection | Bound parameters; whitelisted filters |
| Frontend | Filter UI only |

Exceptions handled

| Type | Name / code | HTTP | Client message |
|---|---|---|---|
| Custom | ApiError | 400 | Start date must be on or before end date |
| Custom | ApiError | 400 | Invalid cursor / companyGroupKey |

## 4.11 Dashboard - /api/dashboard

| Area | Details |
|---|---|
| Validation | limit clamped 1-20 in service |
| SQL injection | Static SQL + params |
| Frontend | Read-only widgets |

Exceptions handled

| Type | Name / code | HTTP | Client message |
|---|---|---|---|
| MySQL / Network | ER_, ECONN | 400/503 | Via errorMapper |

## 4.12 Activity log - /api/activity-log

| Area | Details |
|---|---|
| Validation | activityLogService.queryLogs - dates, module, pagination |
| Sanitization | sanitizeSearchText |
| SQL injection | escapeLike + parameterized |
| Frontend | Read-only table |

Exceptions handled

| Type | Name / code | HTTP | Client message |
|---|---|---|---|
| Custom | ApiError | 403 | Role-based access restrictions |

### 4.13 Stripe public - /api/public/pay

| Area | Details |
|---|---|
| Validation | stripeValidator.js - invoiceType; session_id on result; receipt download (sessionId + token) |
| Frontend | Pay page; publicPayApi.js throws ApiRequestError with errors[] |

Exceptions handled

| Type | Name / code | HTTP | Client message |
|---|---|---|---|
| Stripe | Stripe* types | 4xx | Stripe error message |
| Custom | ApiError | 400 | invoiceType must be regular or xray |
| Custom | ApiError | 400/410 | Invalid / expired payment link |
| Custom | ApiError | 400 | Invoice amount must be greater than zero |

### 4.14 Stripe webhook - /api/webhooks/stripe

Architecture flow: Frontend validates -> Backend validates -> Database (parameterized queries).

| Area | Details |
|---|---|
| Validation | Webhook signature verification |

### 4.15 Public record download - /api/public

Architecture flow: Frontend validates -> Backend validates -> Database (parameterized queries).

| Area | Details |
|---|---|
| Validation | Token validation in recordDownloadService |

## 5. Global exception catalog

### 5.1 Always returned to client (via errorHandler)

| Category | Exact names | Typical HTTP |
|---|---|---|
| Custom | ApiError | 400, 401, 403, 404, 409, 500, 503, 507 |
| JSON parse | SyntaxError (entity.parse.failed) | 400 |
| Upload | MulterError, code LIMIT_FILE_SIZE | 400 |
| Upload | Error message "Unsupported file type" | 400 |
| JWT | TokenExpiredError, JsonWebTokenError, NotBeforeError | 401 |
| MySQL | ER_DUP_ENTRY | 409 |
| MySQL | ER_NO_REFERENCED_ROW_2, ER_ROW_IS_REFERENCED_2, ER_BAD_NULL_ERROR, ER_DATA_TOO_LONG, ER_TRUNCATED_WRONG_VALUE, | |

| |
|---|
| ER_PARSE_ERROR |

| |
|---|
| 400 |

MySQL

```
ER_LOCK_WAIT_TIMEOUT,
ER_LOCK_DEADLOCK
```

503

MySQL

| |
|---|
| ER_ACCESS_DENIED_ERROR, other ER_* |

503 / 400

Network

| |
|---|
| ECONNREFUSED, ECONNRESET, ETIMEDOUT, EHOSTUNREACH, ENOTFOUND, PROTOCOL_CONNECTION_LOST |

503

File system

| |
|---|
| ENOENT, EACCES, EPERM, ENOSPC |

404 / 403 / 507

Stripe

| |
|---|
| Any error.type starting with Stripe |

4xx

Fallback

Unknown Error

500 - "Internal server error"

### 5.2 Server-only (never sent to client)

| Event | Handler |
|---|---|
| unhandledRejection | server.js - logged |
| uncaughtException | server.js - logged |
| Non-critical side effects | runNonCritical / runSafely - logged as warning, request continues |
| Per-item batch failures | orderService.autoCreateOrdersFromBatch, invoiceReminderService - logged per item |
| Stack traces | Logged server-side only |

## 6. SQL injection protection (global)

| Technique | Implementation |
|---|---|
| Named parameters | All models: pool.execute(sql, { :param }) |
| LIKE escaping | sqlSafety.escapeLike, likeContains, model escapeLikePrefix |
| LIMIT safety | assertPositiveInt, Math.min clamps |
| Sort whitelist | Order list: asc / desc only |
| Enum whitelist | Document types, record types, invoice types, report filters |
| No string concat | User input never embedded in SQL text |

## 7. Data sanitization (global)

Architecture flow: Frontend validates -> Backend validates -> Database (parameterized queries).

| Function | Purpose |
|---|---|
| stripControlCharacters | Remove null bytes / control chars |
| sanitizeText | Trim + max length |
| sanitizeSearchText | Search queries (max 200 chars default) |
| escapeHtml | Safe HTML in emails |
| isValidPersonName / isValidOrganizationName | Reject HTML markup and invalid characters in name fields |
| hasHtmlMarkup | Block < / > in free-text fields (notes, CNR reason, cancel reason) |
| fieldLimits.js | Matches DB column sizes |

## 7.1 XSS / script injection (HTML output)

| Surface | Risk | Mitigation |
|---|---|---|
| React UI (frontend/src) | Stored/reflected XSS via user fields | JSX escapes text by default; no dangerouslySetInnerHTML / innerHTML / eval in app code |
| Email HTML (emailService.js, views/emails/*) | Script tags in interpolated values | All interpolated HTML values use escapeHtml; multiline text uses escapeHtmlMultiline |
| API write validators | Script/HTML stored in notes, addresses, search, | |

invoices

addNoHtmlMarkupError on free-text fields;

person/org name patterns on name fields

Storage (sanitizeText)

Bypassed validation / legacy data paths

stripHtmlMarkup removes < and > before persistence

API JSON responses

| |
|---|
| Low (consumed by React, not executed as HTML) |

Content-Type: application/json; security headers on API

Uploaded files

MIME sniffing

Upload middleware whitelists MIME types;
X-Content-Type-Options: nosniff on API and Next.js

Headers (v1.6): X-Content-Type-Options, X-Frame-Options, Referrer-Policy on Express (app.js) and Next.js (next.config.ts).

Name validation (v1.7): Person names allow letters, spaces, -, ', . only. Organization names allow letters, digits, and &.,'()#-/. Enforced in backend validators and matching frontend forms.

System-wide text validation (v1.8): All notes, reasons, descriptions, addresses, memos, and search queries reject angle brackets at validation time. Frontend mirrors the same rules on order, facility, invoice, payment, fax, pickup, and reminder forms.

## 8. Frontend validation utilities (complete coverage v1.3)

Shared module: frontend/src/lib/apiErrorUtils.js

Architecture flow: Frontend validates -> Backend validates -> Database (parameterized queries).
Architecture flow: Frontend validates -> Backend validates -> Database (parameterized queries).

| TwoFactorAuthModal.jsx | Yes (6-digit code) | Yes (code) |
|---|---|---|
| EmployeeFormModal.jsx | Yes | Yes |
| SuspendEmployeeModal.jsx | Yes | Yes (reactivatedDate) |
| orders/new/page.jsx | Yes | Yes (save + provider sync) |
| OrderAddNoteModal.jsx | Yes | Yes |
| OrderNotesModal.jsx | Yes | Yes |
| OrderNotesListModal.jsx | Yes | Yes |
| OrderCancelModal.jsx | Yes | Yes (reason) |
| OrderFaxModal.jsx | Yes | Yes |
| OrderPickupModal.jsx | Yes | Yes |
| SendCopyLetterModal.jsx | Yes | Yes |
| SendInvoiceEmailModal.jsx | Yes | Yes |
| facilities/new/page.jsx | Yes | Yes |
| facilities/[FacilityId]/info/page.jsx | Yes | Yes |
| FacilityAddNoteModal.jsx | Yes | Yes (note) |
| UploadDocumentsModal.jsx | Yes | Yes (file) |
| settings/page.jsx | Yes | Yes |
| ManualPaymentModal.jsx | Yes | Yes |
| CreateInvoiceModal.jsx | Yes | Yes |
| CreateXrayInvoiceModal.jsx | Yes | Yes |
| WriteOffInvoiceModal.jsx | Yes | Yes (amount, orderAction) |
| publicPayApi.js | N/A | Yes (ApiRequestError) |

Status: All identified write surfaces now follow the standard pattern - client validation, disabled submit when invalid, and API field error mapping.

## 9. Validator file index

| File | Modules |
|---|---|
| authValidator.js | Auth |
| employeeValidator.js | Employees |

orderValidator.js

| |
|---|
| Orders (create/update) |

orderActionValidator.js

Order actions

invoiceValidator.js

Invoices

facilityValidator.js

Facilities, notes, documents

providerValidator.js

Providers

paymentValidator.js

Payments

settingsValidator.js

Settings

queryValidators.js

Notifications, order notes, search, payments, payment lists, route id, Stripe result, milestones

stripeValidator.js

Stripe checkout, receipt download

validationHelpers.js

Shared primitives

reportQueryParser.js

Reports

facilityValidation.js

Facility/doctor rules (lib)

## 10. Testing checklist

Architecture flow: Frontend validates -> Backend validates -> Database (parameterized queries).

## 11. Complete backend module checklist (v1.5)

Every API route group and how validation, sanitization, and SQL safety are applied.

| Module | Routes prefix | Validation | Sanitization | SQL injection |
|---|---|---|---|---|
| Auth | /api/auth | authValidator.js (all POST) | Trim identifier; password max length | Employee, AuthSession - :named params |
| Employees | /api/employees | employeeValidator.js (create/update/suspend); validateMilestoneStatsQuery | sanitizeSearchText on list | escapeLikePrefix, parameterized |
| Orders | /api/orders | orderValidator.js, orderActionValidator.js, validateSearchQuery, validateOrderNotesQuery | sanitizeText/sanitizeSearchText; buildOrderDbPayload | Whitelist sort; likeContains; parameterized |
| Invoices | /api/invoices | invoiceValidator.js (writes) | sanitizeTrimOrNull | reportQueryParser; internal ORDER BY |
| Facilities | /api/facilities | facilityValidator.js incl. resolve doctor; validateSearchQuery | sanitizeSearchText, sanitizeText on doctor resolve | escapeLikePrefix; parameterized |
| Providers | /api/providers | providerValidator.js (update); validateSearchQuery | sanitizeSearchText, stripControlCharacters | likeContains; assertPositiveInt |
| Payments | /api/payments | paymentValidator.js, validatePaymentSearchQuery, validatePaymentListQuery | sanitizeTrimOrNull on notes | Parameterized; limit 1-500 on lists |
| Settings | /api/settings | settingsValidator.js | Trim profile fields | Parameterized Employee, EmployeeSettings |
| Notifications | /api/notifications | validateNotificationQuery | sanitizeSearchText | Parameterized Notification |
| Reports | /api/reports | reportQueryParser.js | sanitizeSearchText on all search fields | Bound params; enum whitelists |
| Dashboard | /api/dashboard | Service clamps limit 1-20 | N/A (aggregates) | Static SQL + numeric LIMIT |
| Activity log | /api/activity-log | activityLogService.queryLogs | sanitizeSearchText | escapeLike; parameterized |
| Stripe public | /api/public/pay | stripeValidator.js, validateStripeCheckoutResult, validateStripeReceiptDownload | Token trim in service | Parameterized token/session lookups |
| Stripe webhook | /api/webhooks/stripe | Signature verification | N/A | Parameterized Stripe DB ops |
| Public download | /api/public/records-download | recordDownloadService token check | Token trim | Parameterized :token |
| Health | /health | N/A | N/A | No DB |

## Shared query helpers (queryValidators.js)

| Function | Used for |
|---|---|
| validateNotificationQuery | Notification list limit |
| validateOrderNotesQuery | Order notes pagination/dates |
| validateMilestoneStatsQuery | Employee milestone ISO dates |
| validateSearchQuery | Facility/provider/doctor search (q max 200) |
| validatePaymentSearchQuery | Payment order invoice search |
| validatePaymentListQuery | Manual/online payment lists (orderId, dates, limit) |
| validatePositiveIntRouteParam | Route :id positive integer |
| validateStripeCheckoutResult | Stripe result session_id |
| parsePaymentListLimit | Clamp list results (default 100, max 500) |

## SQL injection rules (all modules)

- 1. Never concatenate user input into SQL strings.
- 2. Always use pool.execute(sql, { :param }) with namedPlaceholders: true.
- 3. LIKE searches: escapeLike, likeContains, or model escapeLikePrefix.
- 4. LIMIT/OFFSET: numeric coercion + Math.min clamp or parsePaymentListLimit.
- 5. ORDER BY / sort: whitelist (asc/desc) or internal column constants only.
- 6. Enums: assertEnum or Set membership before SQL.

DMS Project - Internal technical reference